



Ada[®]

NEWSLETTER

Volume 3, No. 3
September 1986

"Envisaged applications include, but are not limited to, embedded systems applications, ... systems programming, and general EDP applications ... aimed at providing production quality services for ... large projects with many ... programmers."

IBM Ada Product Announcement August 11, 1986.

NOTES FROM THE MANAGING EDITOR

After what seems an eternity International Business Machines Corporation (IBM) Information Systems Group has announced the availability of not one, but two validated Ada programming language compilers for IBM mainframes. If there was doubt in anyone's mind about the Ada programming language and its future those doubts should now be put to rest. Energy previously expended in expressing apprehension about the use of Ada in the Electronic Data Processing applications world can now be re-directed to the more fruitful tasks of engineering adaptable, reliable, maintainable, understandable and reusable software systems in Ada for the full spectrum of applications.

Much to IBM's credit, the mainframe Ada compiler product announcement stresses Ada's capability in systems and general EDP applications development as well as embedded systems work of production quality. The announcement also emphasizes that, "Ada's packages and generic language constructs prove ideal in promoting re-use." IBM appears to fully support it's new Ada products with a "Development System", complete with a full-screen interface to Ada libraries within the library manager.

Our only complaints with this new release from IBM is that the Ada compiler is only able to do 24-bit addressing as opposed to full 32-bit addressing and the product is frightfully expensive when compared to other recent offerings from big blue. One can only hope that both of these situations will improve over time.

There are now (as of September 20, 1986) sixty Ada programming language compilers that have been fully validated by the Department of Defense (DoD) Ada Validation Office (AVO). One can only surmise that the availability of these compilers is the reason for the growing number of companies that are beginning to exploit the commercial Ada marketplace. Math packages, graphics and string handling packages as well as database management systems are at the forefront of this rapidly growing effort by established as well as start-up companies to gain a foot hold on what looks like a mountain of opportunity for re-usable and generic Ada software components. The commercialization of re-usable Ada software components will also create a market for very large storage and retrieval capabilities that will be needed to make this proliferation of components usefully available to the people who need to use them - the software engineers and systems designers. We envision on-line libraries of re-usable Ada components, designs and specifications that will become part and parcel of a system and software

Darrell F. Humphrey
Data Systems
Managing Editor

Robert L. Lewis
Denver Aerospace
Editor

Gary P. McKee

Information & Communication Systems - Denver
Technical Editor

Published by

MARTIN MARIETTA
DATA SYSTEMS
Denver Account

MARTIN MARIETTA
Denver Aerospace



MASSIVELY PARALLEL SYSTEMS AND Ada

by

Jeffrey R. Carter

Senior Engineer--Software
Martin Marietta Denver Aerospace
Mail Stop 8499
Tel. (303) 977-5064

Interest is growing in parallel processing systems--systems with multiple processors which co-operate to solve a problem. Such systems offer dramatic speed increases over single-processor systems. Early parallel systems were dedicated to numeric problems and were called vector processors or array processors.

Recently, parallel systems have been recognized as being useful for general computing. This coincides with advances in technology which make general-purpose parallel systems economically feasible. Several systems are being marketed with hundreds of processors.

As parallel systems move from the experimental arena to actual use, experimenters have been working on the next step in parallel systems; massively parallel systems. Such systems would have thousands to millions of processors. The first such system is now available, but its processors have very limited power.

Such systems could be used as bigger and better versions of currently available parallel systems. A system with one million processors could multiply two 1,000-by-1,000 matrices about as quickly as two-by-two matrices. In addition, massively parallel systems promise to be able to do things which are currently impossible. One can imagine a multi-user system, similar to a Digital Equipment Corporation VAX in size and cost, which can accommodate orders of magnitudes more users than a VAX with no degradation in system response time.

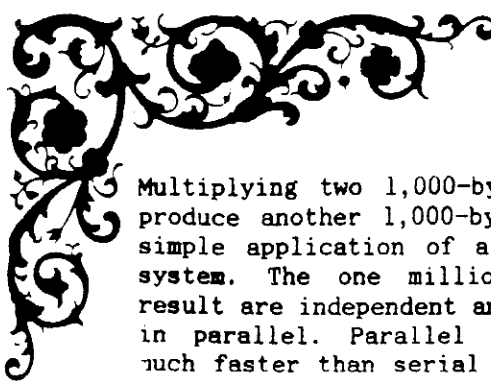
Biological brains are examples of massively parallel systems, so massively parallel systems may be able to model a brain. A branch of artificial intelligence and cognitive research, sometimes called "connectionism," has for several years been using parallel systems to describe neural activity, but have only been able to model small systems. With massively parallel systems, they could model a brain in real time. Connectionist systems model each

neuron-like element as a separate processor. While hardly as complicated as a human brain, which is estimated to have hundreds of billions to trillions of neurons, such a system would be form of true machine intelligence.

As users and supporters of Ada, we may wonder if Ada is suited for use on massively parallel systems. Any language designed for use on massively parallel systems must have certain characteristics. It must be general enough to describe systems in which all or most of the processors execute different processes, all or most of which interact, as well as systems in which all or most of which interact, as well as systems in which all or most of the processors execute copies of the same process on different data. It must provide for describing processes, synchronizing processes, and protection of critical sections of code. It must provide for the dynamic creation of processes.

You will have noticed that Ada provides for all of these features. When all or most of the processors execute different processes, this is described in Ada by many different task objects. Such systems are described as Multiple Instruction, Multiple Data (MIMD) systems. When all or most of the processors execute copies of the same process, this is described in Ada by such structures as arrays of task types. Such systems are described as Single Instruction, Multiple Data (SIMD) systems. Ada's tasking syntax describes processes, synchronization of processes, and protection of critical sections of code. Access types which designate task types allow for dynamic creation of processes.

Parallel systems are available with validated Ada compilers, but there is a problem with using Ada as defined in the Ada Language Reference Manual for many massively parallel applications. A simple SIMD example will serve to illustrate this problem.



Multiplying two 1,000-by-1,000 matrices to produce another 1,000-by-1,000 matrix is a simple application of a massively parallel system. The one million elements of the result are independent and may be calculated in parallel. Parallel processing is very much faster than serial processing for this problem. A parallel matrix multiplication routine in Ada might look like Listing 1.

The problem with this example is that the "*" function must explicitly tell each of the tasks in the MULTER array its position in that array. For a one-million-element matrix, that means one million rendezvous, which must be done serially. This represents a "serial bottleneck" which could seriously degrade the performance of the routine. Similarly, there is a serial bottleneck because "*" must explicitly check that all of the MULTER tasks have terminated before it can return its result. Note that RESULT was not initialized to zero when it was declared, which would have been done serially; instead, each MULTER task initialized its element of RESULT, which is done in parallel.

These problems are hardly major, and can be overcome without modifying the Ada standard. Manufacturers of Ada translators are free to define attributes in addition to those specified in the Ada Language Reference Manual. Therefore, attributes could be defined which would allow a task to determine if it is part of an array, the number of dimensions of that array, the type of the index of each dimension, and the task's index in each of those dimensions. An additional attribute could tell a block whether all tasks declared in that block have terminated. The trouble is, these attributes would not be standardized, and might differ from one implementation to the next. This is a good item to be considered when the Ada standard is next revised, as the addition of attributes to fill these needs would not conflict with the existing standard.

We have seen that massively parallel systems offer benefits in many fields. Ada meets all of the requirements of a language for massively parallel systems, although there is a minor obstacle to Ada's use in massively parallel systems. This obstacle may be overcome within the scope of the existing Ada standard by the implementation of addi-

tional attributes, but such attributes would not be standard. These attributes should be considered in the next revision of the Ada standard. As the Ada standard should not be revised frequently, manufacturers of Ada translators for massively parallel systems should agree on such a standard, in consultation with the Ada Joint Program Office, to avoid problems when the next revision occurs. It appears that current trends in technological progress will not make Ada become obsolete.

Listing 1.

```
package MATRIX_MULTIPLICATION is
    type MATRIX is array (POSITIVE range <>, POSITIVE range <>) of FLOAT;
    function "*" (LEFT : RIGHT : MATRIX) return MATRIX;
    exception MATRIX_SIZE_MISMATCH;
end MATRIX_MULTIPLICATION;

with TEXT_IO;
package body MATRIX_MULTIPLICATION is
    function "*" (LEFT : MATRIX; RIGHT : MATRIX) return MATRIX is
        RESULT : MATRIX (LEFT' RANGE (1), RIGHT' RANGE (2));
        -- for simplicity we will assume that all index ranges begin with
        -- 1
        ALL_TERMINATED : BOOLEAN;

        task type ONE_ELEMENT is
            entry INPUT (ROW : in POSITIVE; COL : in POSITIVE);
        end ONE_ELEMENT;

        type MPS is array (POSITIVE range <>, POSITIVE range <>)
            of ONE_ELEMENT;

        MULTER : MPS (RESULT' RANGE (1), RESULT' RANGE (2));

        task body ONE_ELEMENT is
            MY_ROW : POSITIVE;
            MY_COL : POSITIVE;

        begin -- ONE_ELEMENT

            accept INPUT (ROW : in POSITIVE; COL : in POSITIVE) do
                MY_ROW := ROW;
                MY_COL := COL;
            end INPUT;

            RESULT (MY_ROW, MY_COL) := 0.0;
            SUM_PRODUCTS : for I in LEFT' RANGE (2) loop
                RESULT (MY_ROW, MY_COL) := RESULT (MY_ROW, MY_COL) +
                    LEFT (MY_ROW, I) * RIGHT (I, MY_COL);
            end loop SUM_PRODUCTS;

        exception -- ONE_ELEMENT

            when others =>
                TEXT_IO.PUT_LINE ("Exception in ONE_ELEMENT");
                raise;
```

Continued on next page:

```

end ONE_ELEMENT;

begin --"s"

if LEFT'LENGTH (2) /= RIGHT'LENGTH (1) then
    raise MATRIX_SIZE_MISMATCH;
end if;

MULTIPLY : FOR i IN multer'range (2) LOOP
    for J in MULTER'RANGE (2) loop
        MULTER (I,J).INPUT (I,J);
        --we must identify to each task its row and column
    end loop;
end loop MULTIPLY;

WAIT_FOR_TERMINATION : loop
    --we must explicitly check that all tasks have terminated
    ALL_TERMINATED := TRUE;
    CHECK_TERMINATION : for I in MULTER'RANGE (1) loop
        for J in MULTER'RANGE (2) loop
            if not MULTER (I,J)'TERMINATED then
                ALL_TERMINATED := FALSE;
                exit CHECK_TERMINATION;
            end if;
        end loop;
    end loop;

end loop CHECK_TERMINATION;

exit WAIT_FOR_TERMINATION when ALL_TERMINATED;
delay 0.01; -- or some short period

end loop WAIT_FOR_TERMINATION;

return RESULT;

exception -- "s"

when MATRIX_SIZE_MISMATCH =>
    raise;
when others =>
    TEXT_IO.PUT_LINE ("Exception in *");
    raise;

end "s";

end MATRIX_MULTIPLICATION;

```

Bibliography:

ANSI/MIL-STD-1815A
22 January 1983
Military Standard
Ada Programming Language
Reference Manual

*** Ada Events ***

October

- 1-31 Ada Courses: Management Overview of Ada Technology; Analysis and Design for Ada Software; Advanced Ada Programming Support Workshop; Ada Programming Workshop; Object Oriented Design for Ada
Contact: Roberta Visaggio; EVB Software Engineering, Inc., 5303 Spectrum DR., Frederick, MD 21701 (301) 695-6960
- 15-17 Workshop on Ada Applications in Mission Critical Defense Systems, sponsored by IEEE Computer Society, Jersey Coast Chapter, U.S. Army CECOM, and Monmouth College.
Contact: Howard Hamer; (201)544-2253
- 20 FRONT RANGE Ada WORKING GROUP Fall 1986 Meeting.
Location: Gould, Inc., Denver Technical Center
Contact: Robert L. Lewis, Denver Aerospace (303) 971-6731

- 20-24 NATO's Transition to Ada Conference
Location: SHAPE Technical Centre, The Hague, Netherlands
Contact: Linda Vogelsang; SHAPE Technical Centre; POB 174; 2501 CD The Hague, Netherlands
TEL: 31-7-245550
- 27-30 Computer Society of the IEEE 1986 International Conference On COMPUTER LANGUAGES; Advanced Topics in Ada.
Location: Konoer Hotel, Miami Beach, FL
Contact: IEEE Computer Languages Conference c/o Conference Registrar 1730 Massachusetts Ave., NW Washington, D.C. 20036-1903
Tel: (202) 371-0101
- 28-30 NASA: Workshop on Commonality in Computing for NASA Flight Systems (Commonality - The Future Goal Across NASA Programs)
Location: Johnson Space Center; Houston, TX
Contact: Robert Musgrove; NASA/JSC, EH6, Houston, TX 77058 (713) 483-3566

November

- 12 AIDA: ARTIFICIAL INTELLIGENCE and Ada, 2nd Annual Conference.
Location: George Mason University, Fairfax, VA
Contact: Professor J. Diaz Herrera
Department of Computer and Information Sciences; George Mason University; 4400 University Drive; Fairfax, VA 22030
- 17-21 National SIGAda/AdaJUG joint meeting in conjunction with Ada EXPO '86
Location: Civic Center; Charleston, WV
Contact: Ben Brosgol; Alsays; (617) 890-0030
John Laughlin; Ada Expo '86, POB 868, Frederick, MD 21701; (301) 862-9400

December

- 2-4 Real-Time Systems Symposium
Location: Fairmont Hotel, New Orleans, LA
Contact: Prof. John A. Stankovic; Dept. of Computer Science, Carnegie-Mellon Univ., Pittsburgh, PA; (412) 578-7668
- 9-11 Second ACM SIGSoft/SIGPlan Symposium on Practical Software Development Environments
Location: Palo Alto, CA
Contact: Peter Henderson; Dept. of Computer Science, SUNY at Stony Brook New York 11794; (516) 246-7146 EXT-7090

1987 January

- 7-9 Ada UK 6th International Conference (Ada: The Infrastructure)
Location: University of York, UK
Contact: John Barnes; Alsays, Partridge House, New Town Rd., Hanley on the Thames, RG9 1EN, UK

March

- 16-17 Fourth Washington Ada Symposium in conjunction with Fifth National Conference on Ada Technology
Location: Marriott Crystal Gateway, Arlington, VA
Contact: Al Rodriguez; CECOM, (201) 532-5846

May

- 26-28 Ada Europe Conference (Ada Components: Libraries and Tools)
Location: Stockholm, Sweden
Contact: Prof. Svan Tafvelin; Dept. of Computer Technology; Chalmers Univ. of Technology; S-412, 96 Gothenburg, Sweden

MARTIN MARIETTA